

2024 年非专业级别软件能力认证第一轮

(CSP-J) 入门级 C++语言模拟试题

考生注意事项:

试题纸共有 页, 答题纸共有 8 页, 满分 90 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1. 一个 C++ 程序的执行是从 (A)。

- A. 本程序的 main 函数开始, 到 main 函数结束。
- B. 本程序文件的第一个函数开始, 到本程序文件的最后一个函数结束。
- C. 本程序的 main 函数开始, 到本程序文件的最后一个函数结束。
- D. 本程序文件的第一个函数开始, 到本程序 main 函数结束。

2. 以下代码片段的输出结果为 (C)。

```
#include <iostream>
using namespace std;
int main() {
    int i = 1, j = 1, k = 2;
    if ((j++ || k++) && i++)
        printf("%d %d %d\n", i, j, k);
    return 0;
}
```

- A. 1, 1, 2
- B. 2, 2, 1
- C. 2, 2, 2
- D. 2, 2, 3

3. 将十六进制数 2A 和二进制数 101011 相加, 并将结果转换为十进制数是多少 (D)

- A. 91
- B. 82
- C. 83
- D. 85

4. 现有一段 24 分钟的视频文件, 它的帧率是 30Hz, 分辨率是 1920×1080 , 每帧图像都是 32 位真彩色图像, 使用的视频编码算法达到了 25% 的压缩率。则这个视频文件占用的存储空间大小约是 (C)。

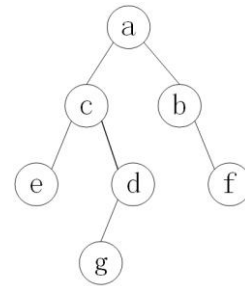
- A. 668GiB
- B. 334GiB
- C. 85GiB
- D. 500GiB

5. 前缀表达式 $* + a b + c d$ 的中缀形式是 (A)。

- A. $(a + b) * (c + d)$
- B. $a + b * c + d$
- C. $a * b + c * d$
- D. $(d + c) * (b + a)$

6. 右图所示的二叉树，其后序遍历的结果是什么？（D）

- A. acedgbf
- B. fbacdge
- C. edgcabf
- D. egdcfba



7. 假设有一组字符 {g,h,i,j,k,l}，它们对应的频率分别为：8%,14%,17%,20%,23%,18%。请问以下哪个选项是字符 g,h,i,j,k,l 分别对应的一组哈夫曼编码？（C）

- A. g: 1100, h: 1101, i: 111, l: 10, k: 00, j: 01
- B. g: 0000, h: 001, i: 010, l: 011, k: 10, j: 11
- C. g: 111, h: 110, i: 101, l: 100, k: 01, j: 00
- D. g: 110, h: 111, i: 101, l: 100, k: 0, j: 01

8. 当 m=7,n=3 时，下列函数返回的结果为（A）

- A. 8
- B. 9
- C. 10
- D. 11

```
int f(int m, int n) {  
    if (m <= 1 || n <= 1) {  
        return 1;  
    }  
    if (m < n) {  
        return f(m, m);  
    } else {  
        return f(m - n, n) + f(m, n - 1);  
    }  
}
```

9. 有 6 个座位连成一排，现有 3 人就坐，则恰好有两个空座位相邻的不同坐法有（C）。

- A. 36 种
- B. 48 种
- C. 72 种
- D. 96 种

10. 以下表述不正确的是（D）。

- A. 在树的层次遍历中以及图论的广度优先遍历中均使用到了队列。
- B. 数组和链表均属于线性表，但是数组的大小是固定的，链表的大小可以动态调整。
- C. 队列和栈是限制性线性表，队列的限制性为后进后出，栈的限制性为后进先出。
- D. 在高精度乘法中运算次数只与两个整数长度较长的那个有关。

11. G 是一个非连通简单无向图，共有 36 条边，则该图至少有（A）个顶点

- A. 10 B. 9 C. 8 D. 7
12. 对于一个有 10 个顶点 25 条边的简单无向图来说，删除（ **C** ）条边后刚好能够让任意点 i 到点 j （ $1 \leq i, j \leq 10$ ）均有路径到达。
- A. 14 B. 15 C. 16 D. 17
13. 计算机的基本存储单位为（ **A** ）。
- A. 字节（byte）
B. 比特（bit）
C. 千字节（kilobyte）
D. 字（word）
14. 某停车场有 7 个连成一排的空车位。现有 3 辆车随机停在这排车位中，则任意两辆车之间至少间隔一个车位的概率为（ **B** ）
- A. 1/5 B. 2/7 C. 6/35 D. 9/35
15. 以下不属于程序编程编程语言的是（ **C** ）。
- A. Java B. Python C. JavaScript D. C++

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

(1)

```
01 #include <iostream>
02 int exGcd(int a, int b, int &x, int &y) {
03     if (!b) {
04         x = 1;
05         y = 0;
06         return a;
07     }
08     int d = exGcd(b, a % b, y, x);
09     y -= (a / b) * x;
10     return d;
11 }
12 int gcd(int a, int b) {
13     while (b != 0) {
14         int temp = b;
15         b = a % b;
16         a = temp;
17     }
18     return a;
19 }
20 int main() {
21     int a, b, x, y;
```

```

22     std::cin >> a >> b;
23     int gcd1 = exGcd(a, b, x, y);
24     int gcd2 = gcd(a, b);
25     std::cout << gcd1 << std::endl;
26     std::cout << (gcd1 == gcd2);
27     return 0;
28 }

```

假设输入的 a, b 均为不超过 2147483647 的自然数, 完成下面的判断题和单选题:

判断题

16. 将 22、25、26 行的 `std::` 删除程序行为不变。 (☒)
17. 程序输出的第二行总是一个 1。 (☒)
18. 当输入 “48 18” 时输出为 “6 1”。 (☒)
19. 当输入 “48 18” 时输出为 “3 0”。 (☒)
20. 此程序的两个函数的功能一样。 (☒)

单选题

21. 当输入 “117 72” 时输出为 (☒)。
- A. “3 1” B. “3 0” C. “9 1” D. “9 0”
22. (4 分) 该程序的时间复杂度为 (☒)。
- A. $O(a)$ B. $O(b)$ C. $O(ab)$ D. $O(\log^{\min(a,b)})$

(2)

```

1  #include <iostream>
2  using namespace std;
3  int n;
4  int a[1000];
5  int f(int x) {
6      int ret = 0;
7      for (; x && x - 1; ) ret++;
8      return ret;
9  }
10 int g(int x) {
11     return x & -x;
12 }
13 int main() {
14     cin >> n;
15     for (int i = 0; i < n; i++) cin >> a[i];
16
17     for (int i = 0; i < n; i++)
18         cout << f(a[i]) + g(a[i]) << ' ';
19     cout << endl;

```

```
20     return 0;
21 }
```

判断题

23. 输入的 n 等于 1001 时，程序不会发生下标越界。（ × ）
24. 输入的 a[i] 必须全为正整数，否则程序将陷入死循环。（ × ）
25. 当输入为“5 2 11 9 16 10”时，输出为“3 4 3 17 5”。（ × ）
26. 当输入为“1 511998”时，输出为“18”。（ √ ）
27. 将源代码中 g 函数的定义（14-17 行）移到 main 函数的后面，程序可以正常编译运行。（ × ）

单选题

28. 当输入为“2 -65536 2147483647”时，输出为（ B ）。
- A. “65532 33” B. “65552 32” C. “65535 34” D. “65554 33”

三、完善程序（单选题，每小题 3 分、合计 30 分）

（1）（寻找图书编码）给定 n 个图书编码和 q 次询问，每次询问包含两个正整数，第一个正整数为需求码的长度，第二个数为需求码，如果有多个需求码，只输出图书编码最小的一个，如果没有需要的需求码，则输出-1。

试补全程序。

```
01 #include <iostream>
02 #include <cmath>
03 #include <limits>
04 using namespace std;
05
06 int main() {
07     int n, q;
08     int a[1010];
09     cin >> n >> q;
10     for (int i = 0; i < n; i++) {
11         cin >> a[i];
12     }
13     int m, d;
14     while (①) {
15         cin >> m >> d;
16         int c, b;
17         bool flage = false;
18         int min = numeric_limits<int>::max();
19         for (int i = 0; i < n; i++) {
20             c = pow(10, m);
21             ②;
```

```

22         if (b == d) {
23             flage = true;
24             if (③) {
25                 ④;
26             }
27         }
28     }
29     if (⑤) {
30         cout << min << endl;
31     } else {
32         cout << "-1" << endl;
33     }
34 }
35 return 0;
36 }

```

29. ①处应填 (**A**)。

A. q-- B. a-- C. n-- D. m--

30. ②处应填 (**B**)。

A. b = a[m] % c
 B. b = a[i] % c
 C. b = a[i] % 100
 D. d = a[i] % c

31. ③处应填 (**C**)。

A. min > b B. min > d C. min > a[i] D. a[i] > min

32. ④处应填 (**D**)。

A. min = i B. c = a[i] C. a[i] = min D. min = a[i]

33. ⑤处应填 (**D**)。

A. !flage B. flage==0 C. 1 D. flage

(2) (泛洪算法) 泛洪算法是一种用于图形的算法，它可以确定图中任意两个节点是否连通，或者找出图中所有连通子图。这个程序实现了图的广度优先搜索 (BFS) 算法，并用它来找到并打印图中的所有连通分量。

```

01 #include <iostream>
02 #include <vector>
03 #include <queue>
04 using namespace std;
05
06 void BFS(vector<vector<int>> &graph, int start, vector<bool>
    &visited) {
07     queue<int> nodes;
08     nodes.push(start);
09     visited[start] = true;

```

```

10     while (!nodes.empty()) {
11         int current = ①;
12         ②;
13         cout << current << " ";
14         for (int neighbor : graph[current]) {
15             if (③) {
16                 visited[neighbor] = true;
17                 ④;
18             }
19         }
20     }
21 }
22
23 void floodFill(vector<vector<int>> &graph, vector<bool> &visited)
24 {
25     for (int i = 0; ⑤; ++i) {
26         if (!visited[i]) {
27             BFS(graph, i, visited);
28             cout << endl;
29         }
30     }
31 }
32 int main() {
33     // 构建图
34     vector<vector<int>> graph = {
35         {1, 2},
36         {0, 2, 3},
37         {1, 3, 4},
38         {2, 4},
39         {3}
40     };
41     vector<bool> visited(graph.size(), false);
42     // 执行泛洪算法
43     floodFill(graph, visited);
44     return 0;
45 }

```

34. ①处应填 (**B**)

- | | |
|----------------------|---------------------|
| A. nodes.back() | B. nodes.front() |
| C. nodes.pop_front() | D. nodes.pop_back() |

35. ②处应填 (**A**)

- | | |
|---------------------|------------------------------|
| A. nodes.pop() | B. nodes.push(current) |
| C. nodes.pop_back() | D. nodes.push_front(current) |

36. ③处应填 (**C**)

A. visited[neighbor]

B. visited[current]

C. !visited[neighbor]

D. !visited[current]

37. ④处应填 (**C**)

A. nodes.push_front(neighbor)

B. nodes.push_back(neighbor)

C. nodes.push(neighbor)

D. nodes.push(current)

38. ⑤处应填 (**A**)

A. $i < \text{graph.size}()$

B. $i \leq \text{graph.size}()$

C. $i < \text{visited.size}()$

D. $i \leq \text{visited.size}()$